Exposio — where the work stands

Short note for our discussion

Author	Esmail Mohammed Salem Abdullah — MSc Cyber Security Engineering
Supervisor	Balázsné Dr. Kail Eszter
Live system	https://exposurescope.vercel.app · source: github.com/SocoSec/exposurescope

1. What exists now

The system is implemented and deployed. The complete path runs in production: a user proves they own an email address by entering a one-time code, the address is checked against breach intelligence, the Breach Risk score is computed on the server, a ranked remediation plan is produced, and an advisory agent answers questions about the result. A live check currently returns 271 breach records for a real address. Nothing is stored at any point.

	Status
API routes / library modules	4 routes · 13 modules (7 domain, 6 platform)
Correctness tests	46 passing — offline, no API keys required
Adversarial tests	18 attacks against a hostile model, 0 escaped
Sensitivity analysis	2,000 identities, parameter sweeps, 6 alternative models
Runtime dependencies	next, react, react-dom — no ORM, no database, no ML runtime
Persistent storage	None. Rate-limit counters only, hashed, expiring within 24 hours

Three commands reproduce all of the above with no credentials and no network access: `npm run selftest`, `npm run redteam`, `npm run sensitivity`.

2. Responding to your three comments

2.1 “The implementation part is superficial”

I have written the implementation up as decisions with reasons and consequences rather than as a description of features. Where state lives given that nothing is stored; why the scan token carries a keyed hash of the address rather than the address; what the password service learns and what it cannot learn; why a source that fails must never be rendered as “no breaches found”; and how the advisory agent is prevented from expressing an incorrect answer rather than asked to avoid one. The long brief sets each of these out with the file and the test that demonstrates it.

2.2 “The scoring model appears largely heuristic”

This was the substantial criticism and I have built the analysis rather than added argument. It runs as `npm run sensitivity` and begins by scoring 2,000 identities twice — once with its own parameterised implementation, once with the deployed scoring code — asserting zero mismatches, so the results provably describe the real model.

Parameter	Effect on ranking	What that means
λ across a 16-fold range	Spearman ≥ 0.90	λ sets the level of the score, not the order
B_max from 0.4 to 1.5	≈ 1.000	A monotone rescaling; a presentation decision
Undated-record age	falls to 0.769	The most consequential free parameter
Tier weights ±30%	0.996, 95% keep band	Conclusions survive the weights being wrong

The last line is the direct answer. I am not claiming 1.0 / 0.7 / 0.4 / 0.2 are the correct weights — they are informed estimates grounded in the account-takeover literature. I am claiming that what a user concludes survives those weights being substantially wrong, and that is measured rather than asserted. The constants are also now given operational meaning: $\lambda = 0.002$/day is a half-life of 347 days, and $B_{max} = 0.8$ defines a score of 100 as one payment-plus-password breach about 112 days old.

I also compared the model against six alternative weighting schemes. Counting breaches ($\rho = 0.641$) and flagging credentials ($\rho = 0.405$) give materially different orderings, so severity weighting and recency decay are each doing real work.

2.3 "The writing feels generic"

I am rewriting the chapters myself from the analysis rather than from a draft. The material in section 3 below is the kind of content I think was missing: specific numbers, a limitation I found in my own model, and errors I found in my own draft.

3. What I found wrong in my own work

3.1 A limitation in the scoring model

Scoring by each person's single worst breach reproduces my full model almost exactly: Spearman $\rho = 0.966$, Kendall τ-b = 0.880, 83.8% band agreement. The reason is concentration — the largest single contributor is a median 92% of the raw score, and for 59% of identities one breach accounts for more than 80% of it. Exponential decay suppresses older records so aggressively that the summation adds little over a maximum.

I would like your view on whether to justify the aggregation by quantifying the cases where the two diverge, adopt the simpler model, or reduce λ so that aggregation does more work.

3.2 Three numerical errors in the draft you reviewed

Location	Issue
§5.3, example 1	Two recency terms are ~100× too large. Raw sum is 0.5123, not 0.563, so the score is 64, not 70. The qualitative point stands and is in fact stronger.
§5.4, example 2	Uses a default recency of 0.05; the implementation assumes 730 days, giving 0.2322. The score is 46 (Medium), not 10 (Low), so the interpretation no longer holds. The text also calls it a live run, which it cannot be.
§5.2 and Table 3	The undated-record assumption is not stated, and three bands are defined where the code has four.

4. A design decision worth discussing

Every comparable service — Have I Been Pwned, DeHashed, the OSINT tools — lets anyone query anyone. Exposio cannot: the scan token carries a keyed hash of the identifier, recomputed and compared on the server, so there is no code path that performs an unverified lookup. This is what makes it a self-assessment instrument rather than an investigative one, and it is why it can be used with study participants at all.

I treated that as a constraint rather than a slogan. Phone lookups were removed from the system because ownership of a number could not be proven, and an unverified phone lookup is a stalking tool. They were restored this week only after SMS verification was implemented, and the property had to be shown to hold across two identifier types: a phone number paired with a token issued for an email address is refused. That work is complete and tested, and is awaiting merge to the deployed branch.

5. What is not implemented

Stated plainly, because the draft describes components that do not exist and I would rather narrow the claims than leave them to be discovered. Not implemented: the GBDT risk model, the BERT-BiLSTM NER pipeline, GNN identity alignment, Shodan integration, the four Exposure Score components beyond Breach Risk, and DSAR automation. A repository search for those terms returns nothing and the only dependencies are next, react and react-dom.

On the first three: the deterministic formula was a deliberate choice, and I would argue a better one here. A published formula can be recomputed by the user, audited by a reviewer and reproduced by a test. A gradient-boosted model removes all three properties, and there is no labelled outcome data to train it against. The interface reflects the narrowed scope: it says “Breach Risk score”, not “Exposure Score”.

6. What I would like to decide tomorrow

1	Whether to keep the summation in the scoring model or adopt the simpler maximum (§3.1).
2	How far to narrow the written claims to match what is built (§5).
3	Whether an expert-ranking exercise is worth running before submission — five to eight practitioners ranking fifteen anonymised profiles, compared with the model by rank correlation. It is roughly a week and would convert “heuristic” into a measured agreement figure.
4	Whether the user study should proceed this term, and what the ethics route is.

Supporting documents, all in the repository: the long implementation brief with the architecture and request-lifecycle figures; IMPLEMENTATION-STATUS.md (component by component); SCORE-MODEL-ANALYSIS.md (the full sensitivity findings); TEST-SPECIFICATION.md (59 test cases mapped to GDPR articles and the OWASP Top 10 for LLM Applications); and STUDY-PROTOCOL.md.